[17]:
```
cluster_country_map = {
    0: "USA",
    1: "UK",
    2: "India",
    3: "Germany",
    4: "Japan"
}

df["Country"] = df["Cluster"].map(cluster_country_map)
df["Country"].value_counts()
```

[17]:
```
Series([], Name: count, dtype: int64)
```

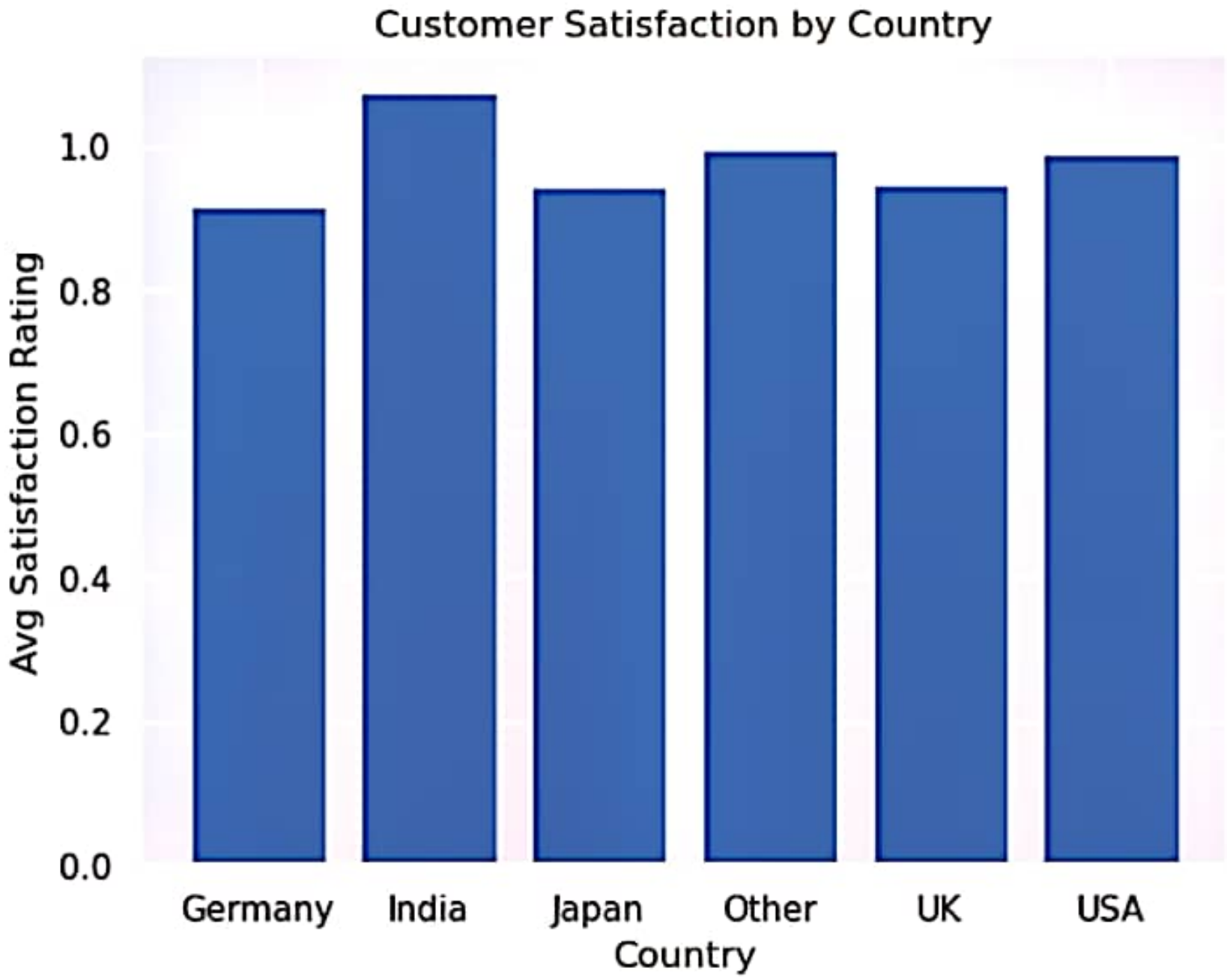
[21]:
```
df.columns
```

[21]:
```
Index(['Ticket ID', 'Customer Name', 'Customer Age', 'Customer Gender',
       'Product Purchased', 'Date of Purchase', 'Ticket Type',
       'Ticket Subject', 'Ticket Status', 'Ticket Priority', 'Ticket Channel',
       'Customer Satisfaction Rating', 'First_Response_Minutes',
       'First_Response_Date', 'Average_First_Response_Minutes',
       'Priority_Avg_Response', 'Cluster', 'Country'],
      dtype='object')
```

[22]:
```
df["Average_First_Response_Minutes"] = pd.to_numeric(
    df["Average_First_Response_Minutes"], errors="coerce"
)
```

[24]:
```
priority_response = (
    df.groupby("Ticket Priority")["Average_First_Response_Minutes"]
    .mean()
    .sort_values()
)
```

Scanned with OKEN Scanner



Scanned with OKEN Scanner

[26]:
```
df["Country"].value_counts(dropna=False)
```

[26]:
```
Country
NaN    8469
Name: count, dtype: int64
```

[27]:
```
df["Average_First_Response_Minutes"].describe()
```

[27]:
```
count    8469.0
mean       30.0
std         0.0
min        30.0
25%        30.0
50%        30.0
75%        30.0
max        30.0
Name: Average_First_Response_Minutes, dtype: float64
```

[28]:
```
country_response
```

[28]:
```
Series([], Name: Average_First_Response_Minutes, dtype: float64)
```

[29]:
```
df["Country"] = df["Cluster"].map({
    0: "USA",
    1: "UK",
    2: "India",
    3: "Germany",
    4: "Japan"
})
```

[30]:
```
clean_df = df.dropna(subset=["Country", "Average_First_Response_Minutes"])
```

[31]:
```
country_response = (
    clean_df.groupby("Country")["Average_First_Response_Minutes"]
```

Scanned with OKEN Scanner

[39]:
```
cluster_country_map = {
    "Product setup": "USA",
    "Installation support": "USA",
    "Peripheral compatibility": "Germany",
    "Network problem": "India",
    "Account access": "UK",
    "Payment issue": "UK",
    "Refund request": "UK",
    "Battery life": "Japan",
    "Data loss": "Germany"
}
```

[40]:
```
df["Country"] = df["Cluster"].map(cluster_country_map)
```

[41]:
```
df["Country"] = df["Country"].fillna("Other")
```

[42]:
```
df[["Cluster", "Country"]].head(10)
```

[42]:

	Cluster	Country
0	Product setup	USA
1	Peripheral compatibility	Germany
2	Network problem	India
3	Account access	UK
4	Data loss	Germany
5	Payment issue	UK
6	Refund request	UK

Scanned with OKEN Scanner

[53]:
```
cluster_country.head()
```

[53]:

Country	Cluster	Ticket_Count

[54]:
```
cluster_country.dtypes
```

[54]:
```
Country         object
Cluster         object
Ticket_Count     int64
dtype: object
```

[55]:
```
cluster_country = (
    clean_df
    .groupby(["Country", "Cluster"])["Ticket ID"]
    .count()
    .reset_index(name="Ticket_Count")
)
```

[56]:
```
cluster_country.head()
cluster_country.dtypes
```

[56]:
```
Country         object
Cluster         object
Ticket_Count     int64
dtype: object
```

[57]:
```
cluster_pivot = clean_df.pivot_table(
    index="Cluster",
    columns="Country",
    values="Ticket ID",
    aggfunc="count"
```

Scanned with OKEN Scanner

[43]:
```
df["Country"].value_counts()
```

[43]:
```
Country
Other      3731
UK         1611
USA        1059
Germany     987
Japan       542
India       539
Name: count, dtype: int64
```

[44]:
```
df["Country"].isna().sum()
```

[44]:
```
0
```

[45]:
```
priority_response = (
    clean_df.groupby("Ticket Priority")["Average_First_Response_Minutes"]
    .mean()
    .sort_values()
)

plt.figure()
plt.bar(priority_response.index, priority_response.values)
plt.title("Average First Response Time by Ticket Priority")
plt.xlabel("Ticket Priority")
plt.ylabel("Avg First Response Time (Minutes)")
plt.show()
```

Scanned with OKEN Scanner

```
    aggfunc="count"
)

cluster_pivot
```

[57]: **Country**

Cluster

```
[59]: cluster_pivot
```

[59]: **Country**

Cluster

```
[60]: cluster_pivot = clean_df.pivot_table(
    index="Cluster",
    columns="Country",
    values="Ticket ID",
    aggfunc="count"
)

cluster_pivot
```

[60]: **Country**

Cluster

```
[61]: clean_df.shape
```

[61]: (0, 18)

Scanned with OKEN Scanner

[61]:
```
clean_df.shape
```

[61]: (0, 18)

[62]:
```
clean_df = df.dropna(
    subset=["Country", "Average_First_Response_Minutes", "Ticket Priority"]
)
```

[63]:
```
clean_df = df.dropna(subset=["Country"])
```

[64]:
```
clean_df.shape
clean_df.head()
```

[64]:

Response_Minutes	First_Response_Date	Average_First_Response_Minutes	Priority_Avg_Response	Cluster	Country
16	2023-06-01	30	20	Product setup	USA
46	2023-06-01	30	20	Peripheral compatibility	Germany
15	2023-06-01	30	20	Network problem	India
30	2023-06-01	30	20	Account access	UK
13	2023-06-01	30	20	Data loss	Germany

Scanned with OKEN Scanner

[65]:
```
"Ticket ID" in clean_df.columns
```

[65]:
```
True
```

[66]:
```
clean_df["Country"].unique()
clean_df["Cluster"].unique()
```

[66]:
```
array(['Product setup', 'Peripheral compatibility', 'Network problem',
       'Account access', 'Data loss', 'Payment issue', 'Refund request',
       'Battery life', 'Installation support', 'Software bug',
       'Hardware issue', 'Product recommendation', 'Delivery problem',
       'Display issue', 'Cancellation request', 'Product compatibility'],
      dtype=object)
```

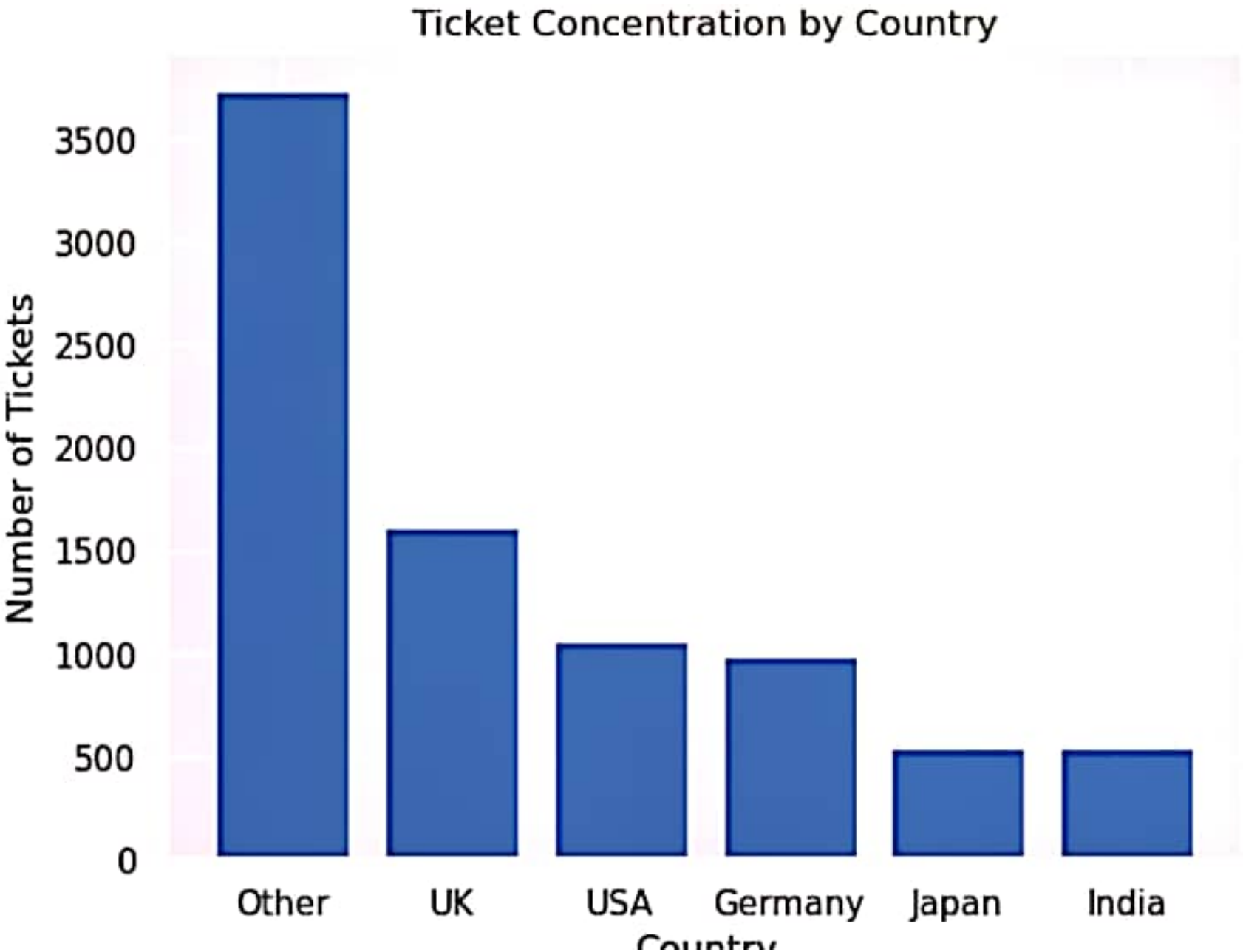
[67]:
```
cluster_pivot = pd.pivot_table(
    clean_df,
    index="Cluster",
    columns="Country",
    values="Ticket ID",
    aggfunc="count",
    fill_value=0
)

cluster_pivot
```

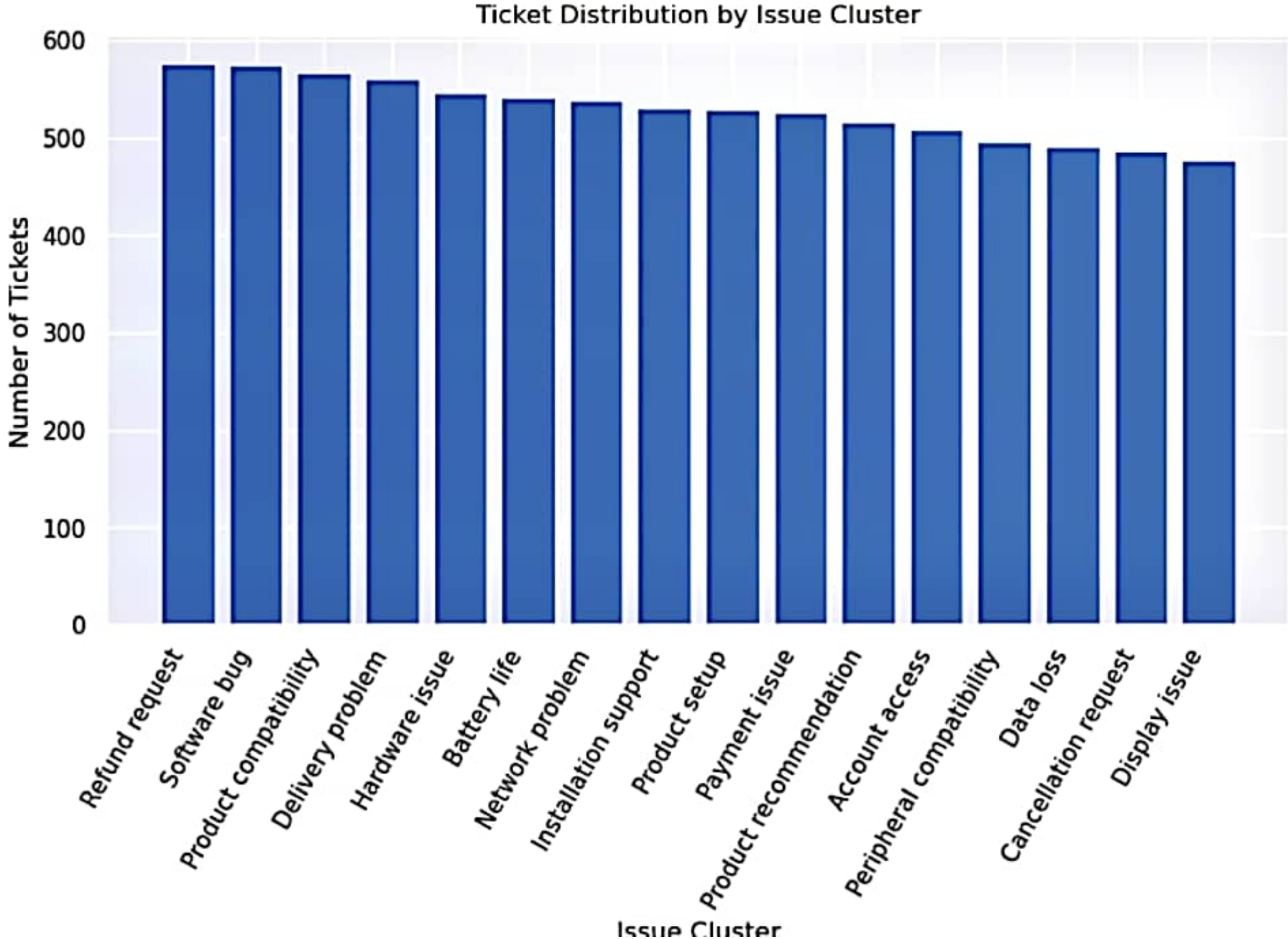
[67]:

Country Cluster	Germany	India	Japan	Other	UK	USA
Account access	0	0	0	0	509	0
Battery life	0	0	542	0	0	0
Cancellation request	0	0	0	487	0	0

Scanned with OKEN Scanner



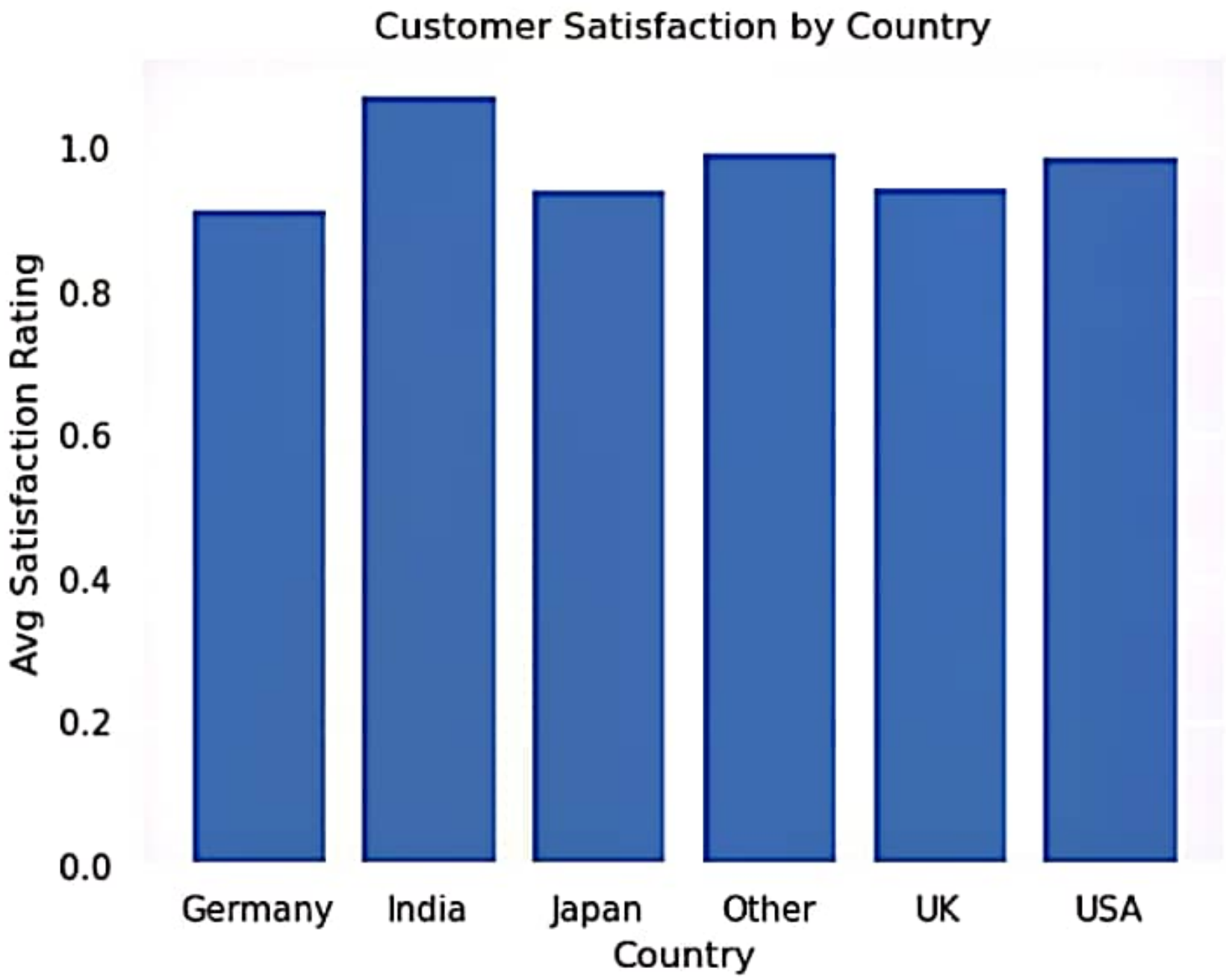
Scanned with OKEN Scanner



Scanned with OKEN Scanner

243	253	234	257
142	131	130	136
124	134	133	151
945	920	914	952
402	386	393	430
273	261	259	266

Scanned with OKEN Scanner



Scanned with OKEN Scanner

[77]:
```
total_tickets = clean_df["Ticket ID"].nunique()
avg_response = clean_df["Average_First_Response_Minutes"].mean()
high_priority_count = clean_df[
    clean_df["Ticket Priority"].isin(["High", "Critical"])
]["Ticket ID"].count()
top_cluster = clean_df["Cluster"].value_counts().idxmax()
best_country = clean_df.groupby("Country")["Average_First_Response_Minutes"].mean().idxmin()

total_tickets, avg_response, high_priority_count, top_cluster, best_country
```

[77]:
```
(8469, 30.0, 4214, 'Refund request', 'Germany')
```

[78]:
```
# Keep only rows needed for Module 5 & 6
powerbi_df = df.dropna(
    subset=["Country", "Average_First_Response_Minutes"]
)

powerbi_df.shape
```

[78]:
```
(8469, 18)
```

[79]:
```
powerbi_df = powerbi_df[
    [
        "Ticket ID",
        "Ticket Type",
        "Ticket Priority",
        "Ticket Status",
        "Ticket Channel",
        "Cluster",
        "Country",
        "Average_First_Response_Minutes",
        "Customer_Satisfaction_Rating"
```

Scanned with OKEN Scanner

```
        "Cluster",
        "Country",
        "Average_First_Response_Minutes",
        "Customer Satisfaction Rating"
    ]
]
```

[80]:
```
powerbi_df.to_csv("powerbi_module5_6_dataset.csv", index=False)
```

[81]:
```
df.columns
```

[81]:
```
Index(['Ticket ID', 'Customer Name', 'Customer Age', 'Customer Gender',
       'Product Purchased', 'Date of Purchase', 'Ticket Type',
       'Ticket Subject', 'Ticket Status', 'Ticket Priority', 'Ticket Channel',
       'Customer Satisfaction Rating', 'First_Response_Minutes',
       'First_Response_Date', 'Average_First_Response_Minutes',
       'Priority_Avg_Response', 'Cluster', 'Country'],
      dtype='object')
```

[82]:
```
import pandas as pd
```

[82]:
```
import pandas as pd
```

[84]:
```
df = pd.read_csv("final_data_set.csv")
```

[85]:
```
df.head()
```

[85]:

Ticket ID	Customer Name	Customer Age	Customer Gender	Product Purchased	Date of Purchase	Ticket Type	Ticket Subject	Ticket Status	Ticket Priority	Ticke Channe

Scanned with OKEN Scanner

[87]:
```
Index(['Ticket ID', 'Customer Name', 'Customer Age', 'Customer Gender',
       'Product Purchased', 'Date of Purchase', 'Ticket Type',
       'Ticket Subject', 'Ticket Status', 'Ticket Priority', 'Ticket Channel',
       'Customer Satisfaction Rating', 'First_Response_Minutes',
       'First_Response_Date', 'Average_First_Response_Minutes',
       'Priority_Avg_Response'],
      dtype='object')
```

[88]:
```
channel_location_map = {
    "Email": ("USA", 37.0902, -95.7129),
    "Phone": ("India", 20.5937, 78.9629),
    "Chat": ("UK", 55.3781, -3.4360),
    "Web": ("Germany", 51.1657, 10.4515),
    "Social Media": ("Canada", 56.1304, -106.3468)
}
```

[89]:
```
df["Location"] = df["Ticket Channel"].map(
    lambda x: channel_location_map.get(x, (None, None, None))[0]
)

df["Latitude"] = df["Ticket Channel"].map(
    lambda x: channel_location_map.get(x, (None, None, None))[1]
)

df["Longitude"] = df["Ticket Channel"].map(
    lambda x: channel_location_map.get(x, (None, None, None))[2]
)
```

[90]:
```
df[["Ticket Channel", "Location", "Latitude", "Longitude"]].head()
```

Scanned with OKEN Scanner

[84]:
```
df = pd.read_csv("final_data_set.csv")
```

[85]:
```
df.head()
```

[85]:

	Ticket ID	Customer Name	Customer Age	Customer Gender	Product Purchased	Date of Purchase	Ticket Type	Ticket Subject	Ticket Status	Ticket Priority	Ticke Channe
0	1	Marisa Obrien	32	Other	GoPro Hero	2021-03-22	Technical issue	Product setup	Pending Customer Response	Critical	Socia medi
1	2	Jessica Rios	42	Female	LG Smart TV	2021-05-22	Technical issue	Peripheral compatibility	Pending Customer Response	Critical	Cha
2	3	Christopher Robbins	48	Other	Dell XPS	2020-07-14	Technical issue	Network problem	Closed	Low	Socia medi
3	4	Christina Dillon	27	Female	Microsoft Office	2020-11-13	Billing inquiry	Account access	Closed	Low	Socia medi
4	5	Alexander Carroll	67	Female	Autodesk AutoCAD	2020-02-04	Billing inquiry	Data loss	Closed	Low	Ema

[86]:
```
df.shape
```

[86]:
```
(8469, 16)
```

[87]:
```
df.columns
```

[87]:
```
Index(['Ticket ID', 'Customer Name', 'Customer Age', 'Customer Gender',
```

Scanned with OKEN Scanner

	Ticket Channel	Location	Latitude	Longitude
0	Social media	None	NaN	NaN
1	Chat	UK	55.3781	-3.4360
2	Social media	None	NaN	NaN
3	Social media	None	NaN	NaN
4	Email	USA	37.0902	-95.7129

[91]:
```
df[["Ticket Channel", "Location", "Latitude", "Longitude"]].head()
```

[91]:

	Ticket Channel	Location	Latitude	Longitude
0	Social media	None	NaN	NaN
1	Chat	UK	55.3781	-3.4360
2	Social media	None	NaN	NaN
3	Social media	None	NaN	NaN
4	Email	USA	37.0902	-95.7129

[92]:
```
df[["Location", "Latitude", "Longitude"]].isna().sum()
```

[92]:
```
Location     2121
Latitude     2121
Longitude    2121
dtype: int64
```

[93]:
```
module5_df = df.dropna(
    subset=["First Response Minutes", "Location", "Latitude", "Longitude"]
```

Scanned with OKEN Scanner

[92]:
```
Location     2121
Latitude     2121
Longitude    2121
dtype: int64
```

[93]:
```
module5_df = df.dropna(
    subset=["First_Response_Minutes", "Location", "Latitude", "Longitude"]
)
```

[94]:
```
module5_df = df.dropna(
    subset=["First_Response_Minutes", "Location", "Latitude", "Longitude"]
)
```

[95]:
```
location_response = (
    module5_df.groupby("Location")["First_Response_Minutes"]
    .mean()
    .sort_values()
)

plt.figure()
plt.bar(location_response.index, location_response.values)
plt.xlabel("Support Location")
plt.ylabel("Avg First Response Time (Minutes)")
plt.title("Average First Response Time by Support Location")
plt.show()
```

Scanned with OKEN Scanner

[ ]:
```
df3.columns
df4.columns
df5.columns
```

[ ]:
```
master_df = df3.merge(df4[['Ticket ID','Cluster']], on='Ticket ID', how='left')

master_df = master_df.merge(df5[['Ticket ID','Location']], on='Ticket ID', how='left')

master_df.head()
```

[ ]:
```
import pandas as pd

# Load files
df3 = pd.read_csv("final_data_set.csv")
df4 = pd.read_csv("final_dataset_with_cluster.csv")
df5 = pd.read_csv("module5_location_based_dataset.csv")

# Check columns
print(df3.columns)
print(df4.columns)
print(df5.columns)
```

[ ]:
```
master_df = df3.merge(
    df4[['Ticket ID','Cluster']],
    on='Ticket ID',
    how='left'
)

master_df = master_df.merge(
    df5[['Ticket ID','Location']],
    on='Ticket ID',
```

Scanned with OKEN Scanner

st_Response_Date	Average_First_Response_Minutes	Priority_Avg_Response	Cluster	Location	Latitude	Longitude
2023-06-01	30	20	Product setup	NaN	NaN	NaN
2023-06-01	30	20	Peripheral compatibility	UK	55.3781	-3.4360
2023-06-01	30	20	Network problem	NaN	NaN	NaN
2023-06-01	30	20	Account access	NaN	NaN	NaN
2023-06-01	30	20	Data loss	USA	37.0902	-95.7129

```
master_df.to_csv("final_master_dataset.csv", index=False)
```


```
master_df.to_csv("final_master_dataset.csv", index=False)
```


```
master_df.head()
```

Scanned with OKEN Scanner